



# 动转静



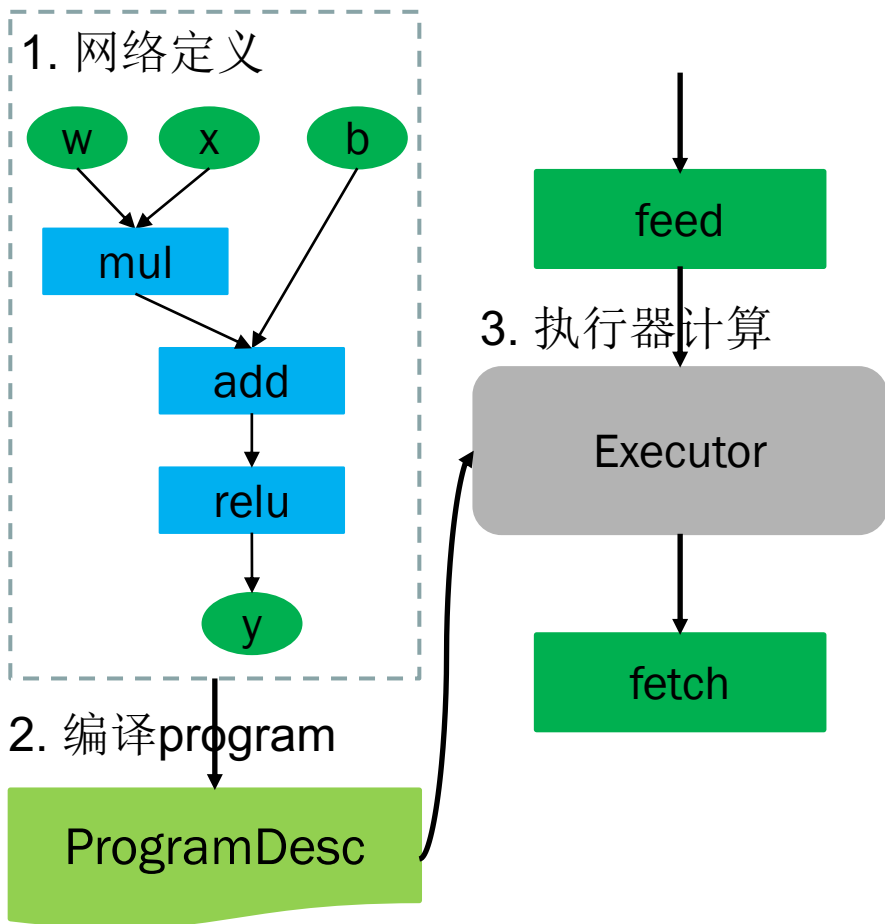
陈特峰  
20210414



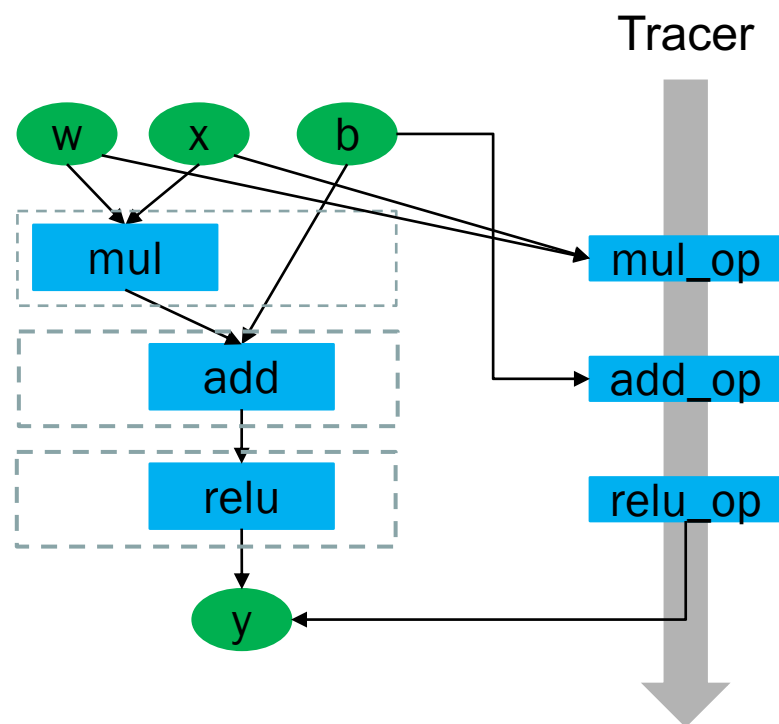
# 目录

- 1. 背景
  - 动静对比、动转静意义
- 2. 整体介绍
  - 实现原理、转换流程
- 3. 类功能设计和关系
  - ProgramTranslator、StaticFunction、DygraphToStaticAst等
  - 抽象语法树及gast使用
- 4. 主要Transformer实现
  - 基础api、容器、控制流、返回等

# 背景 | 静态图 vs 动态图



静态图：运行前编译



动态图：运行时编译

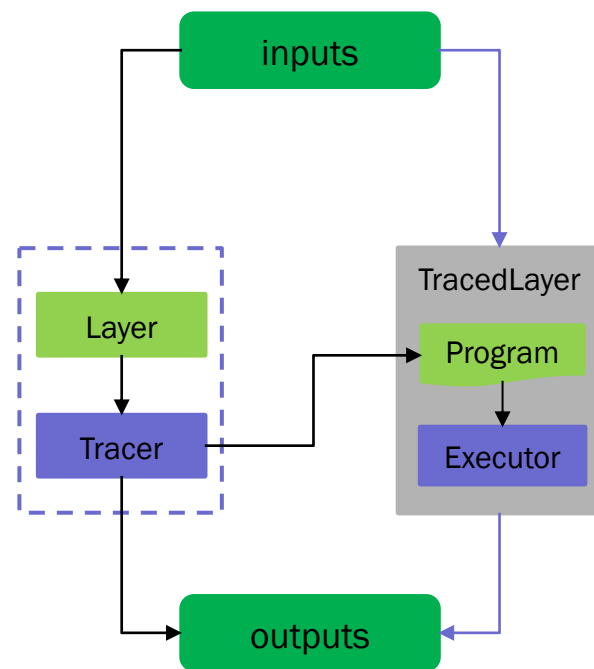
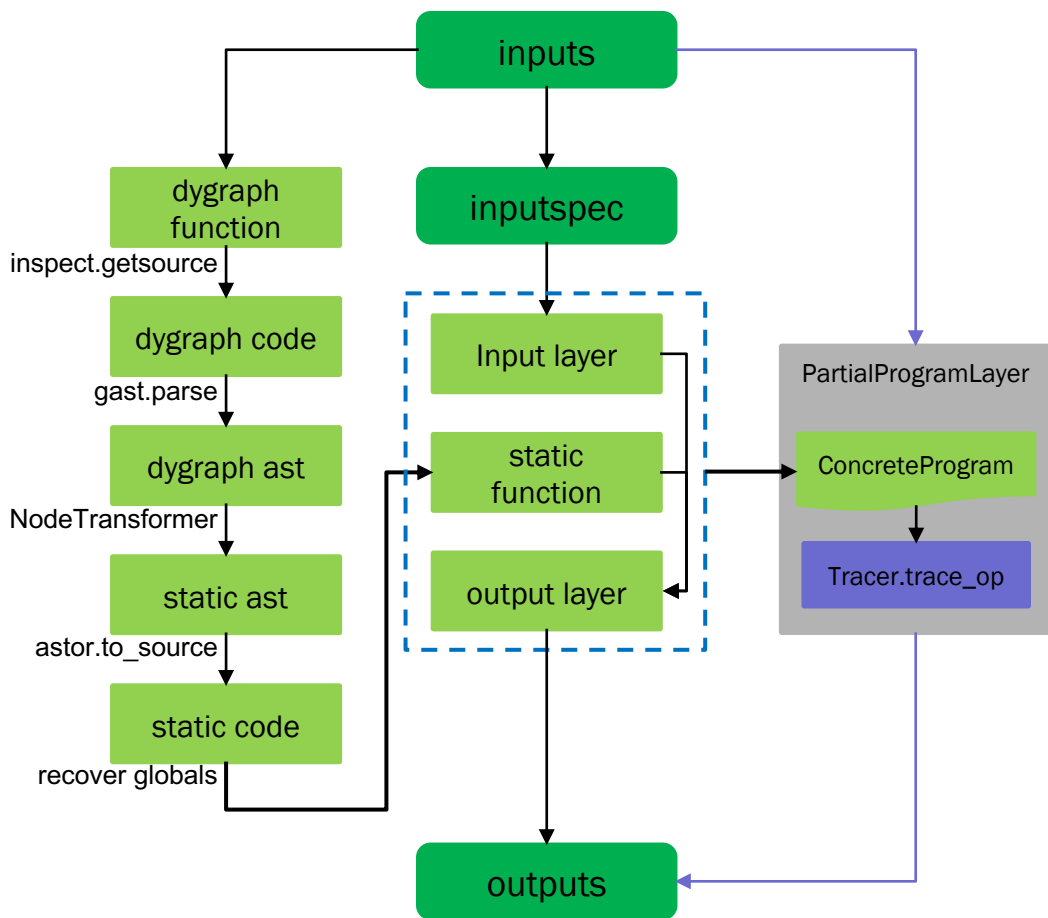
# 背景 | 静态图 vs 动态图

|      | 静态图                     | 动态图                          |
|------|-------------------------|------------------------------|
| 执行机制 | 先编译后执行，预先组网<br>最终才能获得结果 | 即时编译执行，动态添加节点<br>即时获得每个操作的结果 |
| 控制流  | 组网需设计所有可能计算分支           | 与python语法一致                  |
| 训练调试 | 难，图执行完才获得中间结果           | 方便，随时获取变量值                   |
| 性能优化 | 可根据完整计算图进行全局优化          | 只能在算子层面进行局部优化                |
| 部署迁移 | 方便支持多种硬件平台              | 不同平台迁移成本高                    |

动转静意义：结合两者在训练和预测上的优势，方便用户使用

# 整体介绍 | 实现原理

- 箭头表示数据流向或依赖
-  表示转换过程中关联
-  表示转换后关联

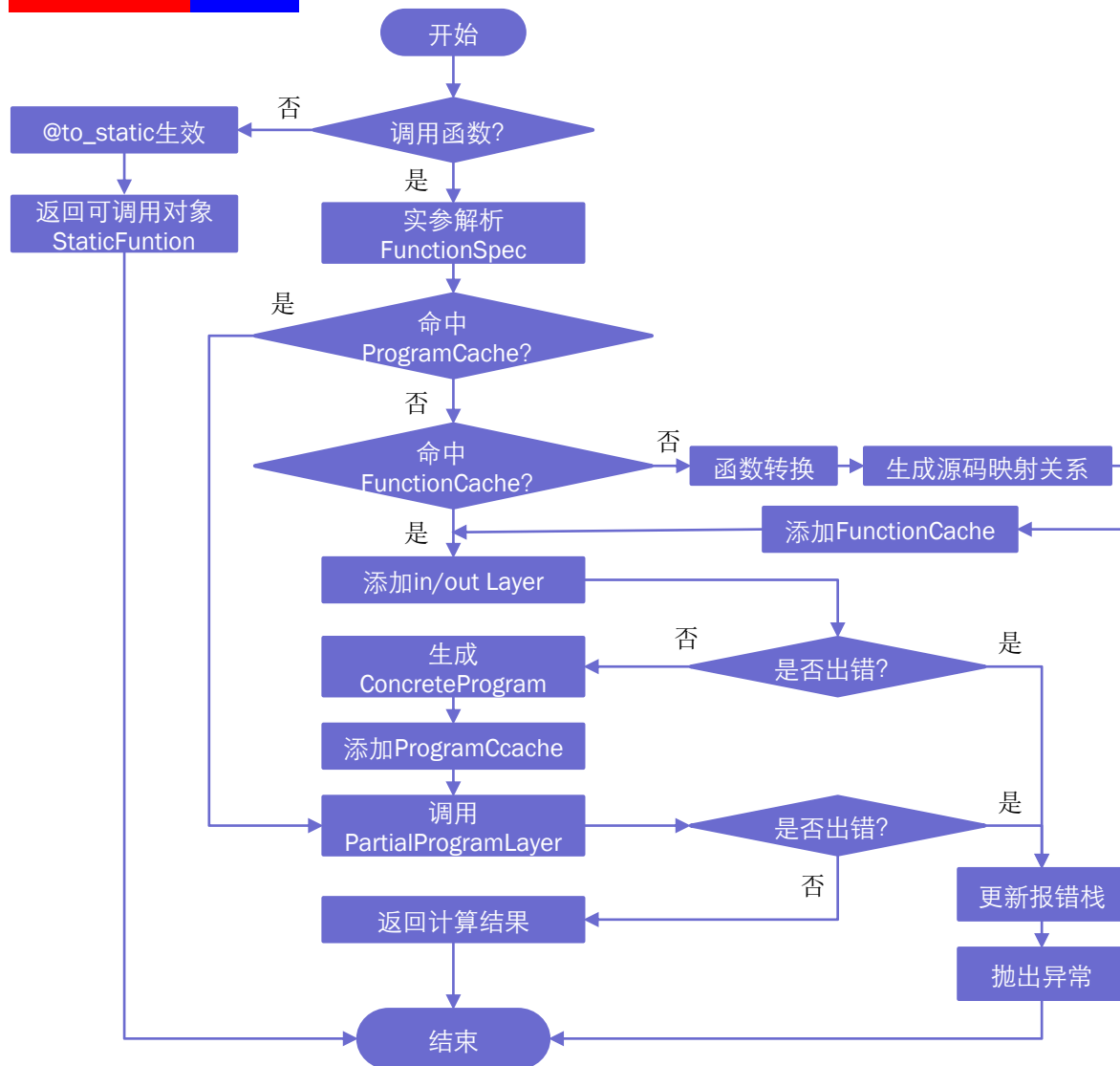


源码转换方法：匹配语法模式，修改ast

Trace方法：运行一次网络，还原program



# 整体介绍 | 转换流程



```
import paddle
```

```
@paddle.jit.to_static
```

```
def func(input_var):
```

```
    # if判断与输入input_var的shape有关
```

```
    if input_var.shape[0] > 1:
```

```
        out = paddle.cast(input_var, "float64")
```

```
    else:
```

```
        out = paddle.cast(input_var, "int64")
```

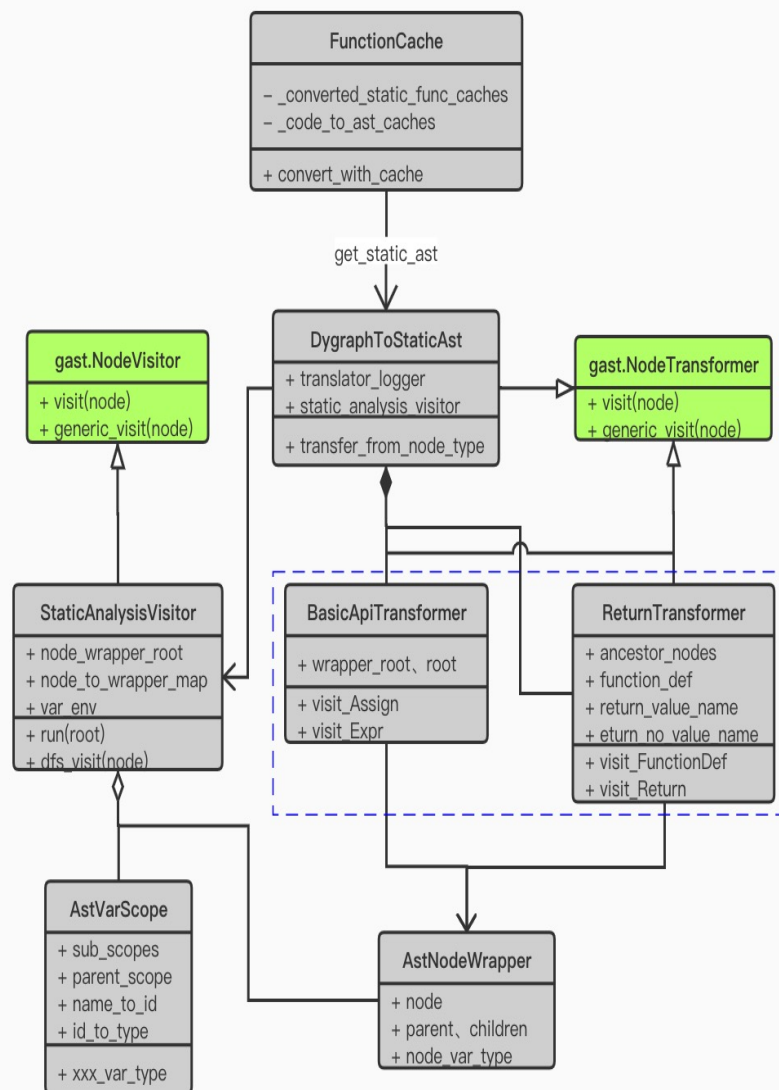
```
    return out
```

- 缓存机制：避免重复组网，加快转换效率
- 转换结果封装成一个大op：在动/静模式下均可继续复用
- 友好性和调试：报错包含原始动态图源码信息、转换后的运行栈



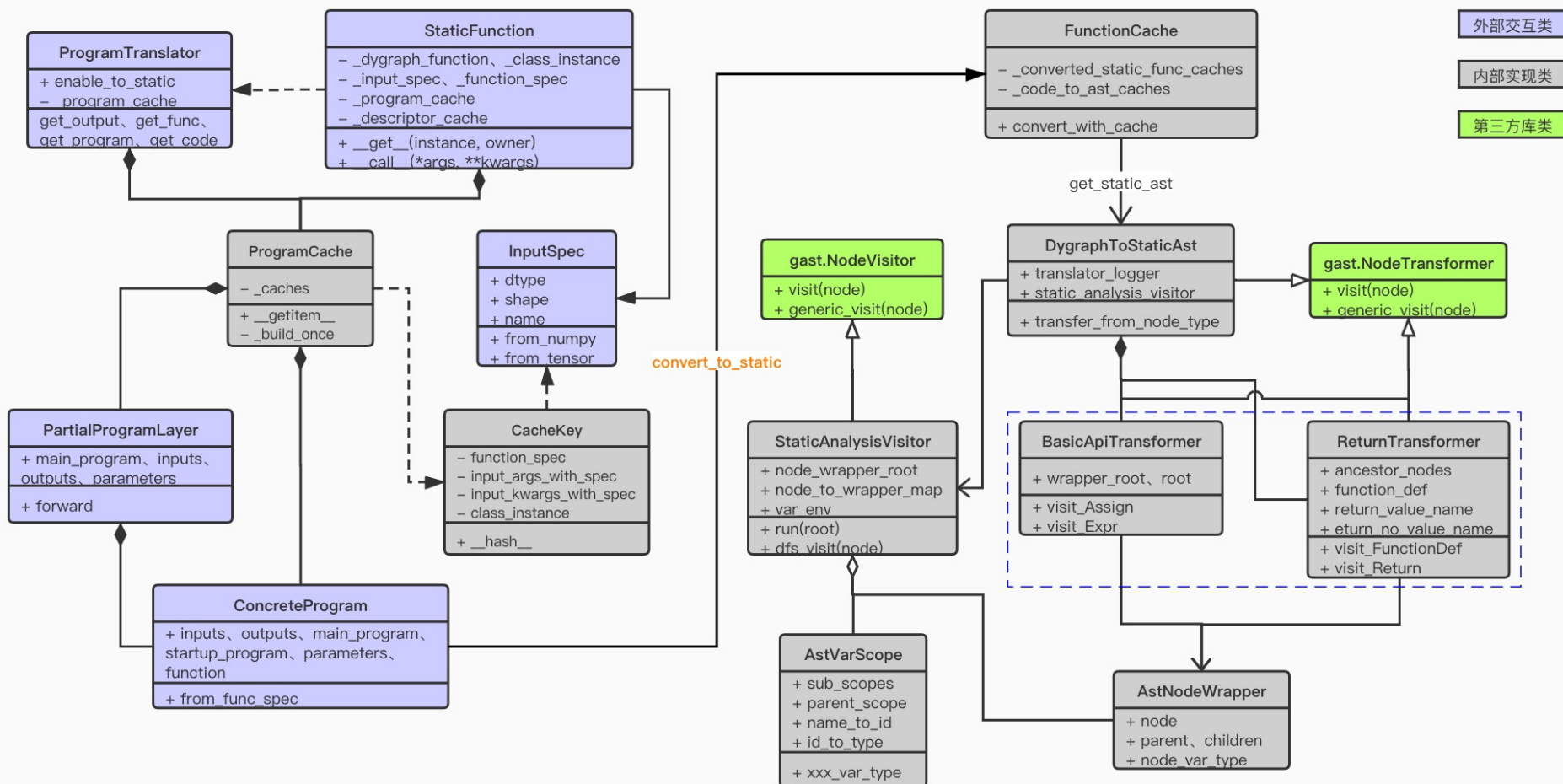
# 类功能设计 | 转换实现类

- **DygraphToStaticAst**: 函数转换逻辑主入口
  - 依次对ast调用自定义的语法转换器
- **gast.NodeTransformer**: gast转换器接口
  - 自定义实现了基础api、控制流、基本容器、断言等（图中仅画出BasicApi和Return）
- **StaticAnalysisVisitor**: 添加节点辅助信息
  - 生成原始ast节点对应wrapper node
  - 维护各层级变量域的变量信息
- **AstNodeWrapper**: ast节点wrapper类
  - 原始节点、父节点、孩子节点、节点类型
- **AstVarScope**: 变量域信息
  - 上/下层变量域引用
  - 变量名->类型





# 类功能设计 | 主要类关系



# 类功能设计 | AST语法

## ● 抽象语法树 (AST)

- 是程序结构的一种高抽象层次的表达，可以认为是与源代码等价的一种抽象表达

## ● py代码执行流程 (CPython)

- 把py源码解析成语法分析树
- 把分析树转化成AST (Abstract Syntax Tree)
- 把AST转化成CFG (Control Flow Graph)
- 基于CFG优化生成bytecode

## ● python的AST定义

- CPython采用抽象语法定义语言 (ASDL) 来描述AST，定义在源码的 parser/python.asdl 中

<https://github.com/python/cpython/blob/master/Parser/Python.asdl>

```
def add(x, y):  
    z = x + y  
    return z
```

→ ast.parse → ast.dump



```
Module(  
    body=[  
        FunctionDef(  
            name='add',  
            args=arguments(  
                args=[  
                    Name(id='x', ctx=Param(), annotation=None, type_comment=None),  
                    Name(id='y', ctx=Param(), annotation=None, type_comment=None)  
                ],  
                posonlyargs=[],  
                vararg=None,  
                kwonlyargs=[],  
                kw_defaults=[],  
                kwarg=None,  
                defaults=[]),  
            body=[  
                Assign(  
                    targets=[  
                        Name(id='z', ctx=Store(), annotation=None, type_comment=None)  
                    ],  
                    value=BinOp(  
                        left=Name(id='x', ctx=Load(), annotation=None, type_comment=None),  
                        op=Add(),  
                        right=Name(id='y', ctx=Load(), annotation=None, type_comment=None)  
                    ),  
                    Return(value=Name(id='z', ctx=Load(), annotation=None, type_comment=None))  
                ],  
                decorator_list=[],  
                returns=None,  
                type_comment=None),  
            type_ignores=[]  
        )  
    ]  
)
```

# 类功能设计 | gast库使用

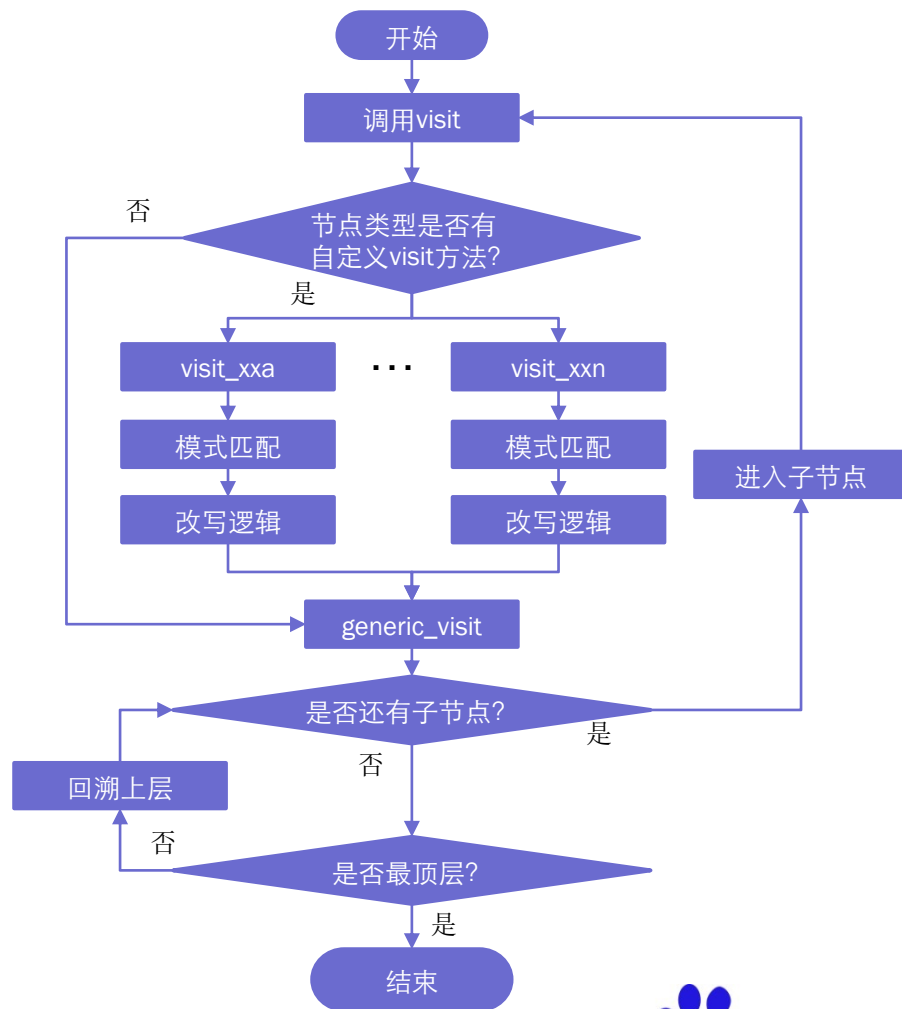
## ● NodeTransformer: 语法树节点修改接口类

- **visit(self, node):** 返回值为替换节点 (None则表示删除该节点); 默认逻辑是根据节点类型调用self.visit\_classname方法更新节点, 若未定义则调用generic\_visit
- **generic\_visit(self, node):** 获取子节点列表, 依次对其调用visit方法进行更新
- **visit\_classname(self, node):** 自定义classname类型的更新方法

```
def add(x, y):  
    z = x + y  
    return z
```

```
def sub(x, y):  
    z = x - y  
    return z
```

```
class ExampleTransformer(gast.NodeTransformer):  
    def visit_FunctionDef(self, node):  
        self.generic_visit(node)  
        if node.name == 'add':  
            node.name = 'sub'  
        return node  
  
    def visit_BinOp(self, node):  
        self.generic_visit(node)  
        if isinstance(node.op, gast.Add):  
            node.op = gast.Sub()  
        return node
```



# Transformer | 支持语法概览

- 动态图能够即时获取运行结果，与python语法兼容，语义表达能力比静态图更丰富，当前paddle在api层次已经做了动静兼容，两者差异主要表现在与tensor相关的运算逻辑中，包括控制流、运算、shape获取等，因此动转静的主要工作也在于此，下面简列支持的语法分类：

| 语法分类    | Transformer                                                                                                                                           |
|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| 基础api   | BasicApiTransformer                                                                                                                                   |
| 变量shape | TensorShapeTransformer                                                                                                                                |
| 基本容器    | ListTransformer                                                                                                                                       |
| 控制流     | ForLoopTuplePreTransformer、ForToWhileTransformer、BreakTransformOptimizer、BreakContinueTransformer、LoopTransformer、ReturnTransformer、IfElseTransformer |
| 运算      | LogicalTransformer、CastTransformer                                                                                                                    |
| 函数      | PrintTransformer、CallTransformer                                                                                                                      |

# 基础api | BasicApiTransformer

- 匹配句式：（1） to\_variable、to\_tensor （2） CosineDecay等Decay类
- 搜索位置：（1） 赋值语句右值 （2） 独立表达式
- 转换逻辑：
  - 句式（1）转换为paddle.assign，它是静态图由numpy值创建placeholder的接口
  - 句式（2）转换为fluid.Layers下相应函数

```
def dyfunc_to_variable(x):  
    res = dygraph.to_variable(  
        value=np.zeros(shape=1, dtype=np.int32))  
    return res
```

```
def dyfunc_to_variable(x):  
    res = paddle.assign(  
        x=np.zeros(shape=1, dtype=np.int32))  
    return res
```

```
def dyfunc_CosineDecay():  
    base_lr = 0.1  
    CosineDecay = fluid.dygraph.CosineDecay(  
        learning_rate=base_lr,  
        step_each_epoch=10000, epochs=120)  
    lr = CosineDecay()  
    return lr
```

```
def dyfunc_CosineDecay():  
    base_lr = 0.1  
    lr = fluid.layers.cosine_decay(  
        learning_rate=base_lr,  
        step_each_epoch=10000, epochs=120)  
    return lr
```

# 变量shape | TensorShapeTransformer

- 匹配句式：形如x.shape或x.shape[0]出现在如下语句时：
  - (1) 赋值语句的右值 (2) paddle api的参数 (3) 控制流中，if或while判断条件、for的迭代表达式
- 转换逻辑：改写为通过函数获取shape结果，其将根据x是否为tensor选择nn.shape或直接x.shape
  - 变量作为赋值语句右值出现时：改写为从convert\_var\_shape\_simple获取，并保存在额外创建的变量\_\_static\_convert\_var\_shape\_suffix中
  - 变量首次出现在参数、控制流条件中时改写为convert\_var\_shape代替，变量已经上文中出现则通过：eval\_if\_exist\_else\_none、choose\_shape\_attr\_or\_api、convert\_var\_shape\_simple配合使用间接获取\_\_static\_convert\_var\_shape\_suffix的变量结果

```
def dyfunc_tensor_shape_1(x):  
    x = fluid.dygraph.to_variable(x)  
    res = fluid.layers.reshape(x, shape=x.shape)  
    return res
```

```
def dyfunc_tensor_shape_1(x):  
    x = paddle.assign(x)  
    res = fluid.layers.reshape(x,  
                               shape=paddle.jit.dy2static.  
                               convert_var_shape(x, in_control_flow=False))  
    return res
```

```
def dyfunc_tensor_shape_5(x):  
    x = fluid.dygraph.to_variable(x)  
    s = x.shape[0]  
    res = fluid.layers.reshape(x, shape=(-1, s))  
    return res
```

```
def dyfunc_tensor_shape_5(x):  
    x = paddle.assign(x)  
    s = x.shape[0]  
    s__static_convert_var_shape_suffix_0 = (paddle.jit.dy2static.  
    convert_var_shape_simple(x)[0])  
    res = fluid.layers.reshape(x, shape=(-1, paddle.jit.dy2static.  
    choose_shape_attr_or_api(s, paddle.jit.dy2static.  
    eval_if_exist_else_none('s__static_convert_var_shape_suffix_0',  
    locals()))))  
    return res
```

# 基本容器 | ListTransformer

- 匹配句式：（1）list或dict pop （2）list append
- 搜索位置：（1）所有位置的pop （2）append出现在依赖tensor的控制流（if/while/for）区块中
- 转换逻辑：（1）pop用convert\_op动态函数替换，其将根据对象是否为tensor调用相应pop方法 （2）append出现在匹配位置时，先后对其list对象的操作做如下转换：
  - a) 以paddle.tensor.array\_write替换append操作
  - b) 以paddle.tensor.create\_array(dtype= 'float32 ')替换初始化定义
  - c) 以paddle.tensor.array\_write替换下标访问修改

```
def test_list_pop_in_if(x):  
    x = fluid.dygraph.to_variable(x)  
    a = []  
    if x.numpy()[0] > 0:  
        a.append(x)  
        a.append(fluid.layers.fill_constant(  
            shape=[1], value=1, dtype='int64'))  
    else:  
        a.append(x + 1)  
        a.append(fluid.layers.fill_constant(  
            shape=[2], value=2, dtype='int64'))  
    item1 = a.pop(1)  
    return item1
```

```
def test_list_pop_in_if(x):  
    x = paddle.assign(x)  
    a = paddle.tensor.create_array(dtype='float32')  
    if x.numpy()[0] > 0:  
        paddle.tensor.array_write(x=x, i=paddle.tensor.array_length(a), array=a)  
        paddle.tensor.array_write(  
            x=fluid.layers.fill_constant(shape=[1], value=1, dtype='int64'),  
            i=paddle.tensor.array_length(a), array=a)  
    else:  
        paddle.tensor.array_write(x=x + 1, i=paddle.tensor.array_length(a),  
            array=a)  
        paddle.tensor.array_write(x=fluid.layers.fill_constant(shape=[2],  
            value=2, dtype='int64'), i=paddle.tensor.array_length(a), array=a)  
    item1 = paddle.jit.dy2static.convert_pop(a, 1)  
    return item1
```



# 循环 | For循环预处理

- **ForLoopTuplePreTransformer:** for迭代赋值表达式中包含tuple，则解包改写为朴素格式，支持如下三种形式转换：

```
for var1, var2 in var:  
    x += 1
```

```
for __for_loop_iter_var_0 in var:  
    var1 = __for_loop_iter_var_0[0]  
    var2 = __for_loop_iter_var_0[1]  
    x += 1
```

```
for i, (var1, var2) in enumerate(var):  
    x += 1
```

```
for i, __for_loop_iter_var_0 in enumerate(var):  
    var1 = __for_loop_iter_var_0[0]  
    var2 = __for_loop_iter_var_0[1]  
    x += 1
```

```
for t in enumerate(var):  
    x += 1
```

```
for __for_loop_var_index_0, __for_loop_iter_var_0 in enumerate(var):  
    t = __for_loop_iter_tuple_index_0, __for_loop_iter_var_0  
    x += 1
```

- **ForToWhileTransformer:** 将for循环改写为等价的while表达

1. for语义是迭代遍历集合元素，因此可统一拆解为init\_stmts（下标索引、元素变量初始化），cond\_stmt（通过下标索引判断是否迭代结束），body\_stmts（循环处理逻辑）三部分语句
2. 以cond\_stmt、body\_stmts组成while区块，并在其起始前添加init\_stmts语句

```
def test_for_to_while(x):  
    x = paddle.assign(x)  
    var = [1, 2]  
    for t in var:  
        x += t  
    return x
```

```
def test_for_to_while(x):  
    x = paddle.assign(x)  
    var = [1, 2]  
    __for_loop_var_index_0 = 0  
    __for_loop_var_len_0 = paddle.jit.dy2static.convert_len(var)  
    while __for_loop_var_index_0 < __for_loop_var_len_0:  
        x += __for_loop_iter_var_0  
        __for_loop_var_index_0 += 1  
    return x
```



# 循环 | BreakTransformOptimizer

- 匹配句式: for/while循环紧接if ... break
- 转换逻辑:
  - for循环先转换为等价的while
  - 将break的判断条件上提添加到while判断条件中, 删除if语句块

```
def test_optim_break_in_for(x):  
    x = paddle.to_tensor(x)  
    for i in range(10):  
        if x.sum() > 5:  
            break  
        x += 10086  
    x += i  
    if i < 3:  
        x = x * 2  
    return x
```

```
def test_optim_break_in_for(x):  
    x = paddle.assign(x)  
    i = 0  
    while i < 10 and not x.sum() > 5:  
        x += i  
        if i < 3:  
            x = x * 2  
        i += 1  
    return x
```

# 循环 | BreakContinueTransformer

- 匹配句式：包含break/continue的for/while循环
- 转换逻辑：
  1. 额外创建bool变量\_（以\_\_break或\_\_continue为前缀）用来判断是否走到break/continue分支
  2. break/continue出现处添加对变量赋真的赋值语句，同一区块内、在它们位置之后的语句直接删除
  3. break/continue上层区块、在它们位置之后添加if not表示未走入break/continue分支
  - 4.1 包含break句式：若为for循环先转化为while；在循环中添加not \_\_break判断条件，并在循环起始之前添加\_\_break变量的初始化定义
  - 4.2 包含continue句式在循环体起始处添加\_\_continue变量的初始化定义

```
def test_break_in_while(x):  
    x = fluid.dygraph.to_variable(x)  
    i = fluid.layers.fill_constant(  
        shape=[1], dtype='int32', value=0)  
    while i < 10:  
        i += 1  
        if i > 5:  
            break  
        x += 10086  
        x += i  
    return x
```



```
def test_break_in_while(x):  
    x = paddle.assign(x)  
    i = fluid.layers.fill_constant(  
        shape=[1], dtype='int32', value=0)  
    __break_0 = paddle.fluid.layers.fill_constant(  
        shape=[1], dtype='bool', value=False)  
    while i < 10 and not __break_0:  
        i += 1  
        if i > 5:  
            __break_0 = paddle.fluid.layers.fill_constant(  
                shape=[1], dtype='bool', value=True)  
            if not __break_0:  
                x += i  
    return x
```

# 循环 | LoopTransformer

## ● for/while转换过程:

1. 分析有效变量 (NameVisitor): 分析循环上下文变量, 求解出在循环cond、body访问的变量列表, 以及需要在循环前预定义的列表 (body中先读后写、被循环后使用)
2. 改写为循环前变量预定义、退出判断、处理逻辑、迭代调度四部分, 其中退出判断和处理逻辑两部分定义在新函数中, 参数由NameVisitor分析结果确定, 最后函数对象作为参输入入 convert\_while\_loop由其进行迭代调度, 其将根据是否依赖tensor而添加[while op算子](#)到静态图中或直接使用python解释执行

```
def for_loop_dyfunc2(max_len):
    x = fluid.layers.fill_constant(
        shape=[1, 2], dtype='int32', value=1)
    for i in range(max_len):
        if i > 1:
            s = a
        a = 1
    ret = fluid.layers.fill_constant(
        shape=[1], dtype='int32', value=s)
    return ret
```

```
def for_loop_dyfunc2(max_len):
    x = fluid.layers.fill_constant(shape=[1, 2], dtype='int32', value=1)
    a = paddle.jit.dy2static.data_layer_not_check(name='a', shape=[-1],
        dtype='float32')
    s = paddle.jit.dy2static.data_layer_not_check(name='s', shape=[-1],
        dtype='float32')
    i = 0

    def for_loop_condition_0(i, max_len, s, a):
        return i < max_len

    def for_loop_body_0(i, max_len, s, a):
        if i > 1:
            s = a
        a = 1
        i += 1
        return i, max_len, s, a

    [i, max_len, s, a] = paddle.jit.dy2static.convert_while_loop(
        for_loop_condition_0, for_loop_body_0, [i, max_len, s, a])
    ret = fluid.layers.fill_constant(shape=[1], dtype='int32', value=s)
    return ret
```

# 返回 | ReturnTransformer

- 匹配句式：包含多个return语句的函数

- 转换逻辑：

1. 确定函数内所有return语句的最长返回值个数max\_len，额外创建一个变量\_\_return\_value（max\_len大于1时以tuple打包）表示最后的返回值
2. 对于每个return语句做如下转换：（1）返回值按序赋值给\_\_return\_value，若返回值个数小于max\_len，则用添加占位符变量\_\_no\_value\_return\_var补齐，删除return语句（2）同一分支之后的语句无效，直接删除（3）若处于为if区块内，则额外创建bool变量\_\_return表示是否进入该分支，并在if之前添加初始化\_\_return初始化语句；回退上层区块若无else分支则将后续内容用if not \_\_return包裹（4）若在循环中，则将\_\_return变量添加循环判断条件中（for先转换为while）
3. 函数起始处添加\_\_return\_value、占位符变量\_\_no\_value\_return\_var初始化定义

```
def test_return_if(x):  
    x = fluid.dygraph.to_variable(x)  
    y = x + 1  
    if x < 0:  
        return -x, y  
        x -= 1  
    x += 3  
    return x
```



```
def test_return_if(x):  
    __no_value_return_var_0 = paddle.fluid.layers.fill_constant(shape=[1],  
        dtype='float64', value=1.77113e+279)  
    __return_value_init_0 = paddle.fluid.layers.fill_constant(shape=[1],  
        dtype='float64', value=0.0)  
    __return_value_init_1 = paddle.fluid.layers.fill_constant(shape=[1],  
        dtype='float64', value=0.0)  
    __return_value_0 = __return_value_init_0, __return_value_init_1  
    x = paddle.assign(x)  
    y = x + 1  
    __return_0 = paddle.jit.dy2static.create_bool_as_type(x < 0, False)  
    if x < 0:  
        __return_0 = paddle.jit.dy2static.create_bool_as_type(x < 0, True)  
        return_value_0 = -x, y  
    if not __return_0:  
        x += 3  
        __return_1 = paddle.jit.dy2static.create_bool_as_type(not  
            __return_0, True)  
        return_value_0 = x, __no_value_return_var_0  
    return __return_value_0
```

# 逻辑运算 | LogicalTransformer

- 匹配句式: Not、以convert\_share\_shape起始的Compare (<、> == !=) 、And、Or
- 转换逻辑: 分别以convert\_logical\_not、convert\_shape\_compare、convert\_logical\_and、convert\_logical\_or函数替换, 这些函数会在组网时根据其中元素是否为tensor或是python变量确定不同的比较逻辑

```
def test_logical_not_and_or(x):  
    x = paddle.to_tensor(x)  
    a = 1  
    if x < 10 and (a < 4 or a > 0) or a < -1 or not x > -1:  
        x = x - 1  
    else:  
        x = x + 1  
    return x
```

```
def test_logical_not_and_or(x):  
    x = paddle.assign(x)  
    a = 1  
    if paddle.jit.dy2static.convert_logical_or(lambda : paddle.jit.  
        dy2static.convert_logical_or(lambda : paddle.jit.dy2static.  
        convert_logical_and(lambda : x < 10, lambda : paddle.jit.dy2static.  
        convert_logical_or(lambda : a < 4, lambda : a > 0)), lambda : a <  
        -1), lambda : paddle.jit.dy2static.convert_logical_not(x > -1)):  
        x = x - 1  
    else:  
        x = x + 1  
    return x
```

```
def test_shape_equal(x):  
    x = paddle.to_tensor(x)  
    y = paddle.zeros([1, 2, 3])  
    if x.shape == y.shape:  
        pass  
    return x
```

```
def test_shape_equal(x):  
    x = paddle.assign(x)  
    y = paddle.zeros([1, 2, 3])  
    if paddle.jit.dy2static.convert_shape_compare(paddle.jit.dy2static.  
        convert_var_shape(x, in_control_flow=True), '==', paddle.jit.  
        dy2static.convert_var_shape(y, in_control_flow=True)):  
        pass  
    return x
```

# 条件 | IfElseTransformer

## ● if-else转换过程:

1. 获取不同域变量列表 (NameVisitor): 给定语法树入口节点, 获取遍历路径指定区间 (未指定则整棵子树) 的所有变量名
2. 由NameVisitor获取if-else之前、if区块内、else区块内, if-else之后的变量列表, 并根据变量访问模式 (读/写) 以及出现位置, 确定条件语句执行结果存储变量、定义在区块前并被分支内修改的、需要在语句前预先定义的变量列表
3. 统一改写为前置变量预定义、条件为真处理函数、条件为假处理函数、执行调度四部分, 其中处理函数参数为条件语句前定义并且首次访问是读模式的变量, 最后函数对象作为参数传入convert\_ifelse由其完成条件分支调度执行, 其将根据判断条件是否依赖tensor选择添加[cond算子](#)到静态图中或直接使用python解释执行

```
def dyfunc_with_if_else3(x):  
    y = x + 1  
    if fluid.layers.mean(x).numpy()[0] < 5:  
        x = x + 1  
        z = x + 2  
        q = x + 3  
    else:  
        y = y + 1  
        z = x - 2  
        m = x + 2  
        n = x + 3  
    q = q + 1  
    n = q + 2  
    x = n  
    return x
```

```
def dyfunc_with_if_else3(x):  
    y = x + 1  
    q = paddle.jit.dy2static.data_layer_not_check(  
        name='q', shape=[-1], dtype='float32')  
    z = paddle.jit.dy2static.data_layer_not_check(  
        name='z', shape=[-1], dtype='float32')  
  
    def true_fn_0(x, y):  
        x = x + 1  
        z = x + 2  
        q = x + 3  
        return q, x, y, z  
  
    def false_fn_0(x, y):  
        y = y + 1  
        z = x - 2  
        m = x + 2  
        n = x + 3  
        return q, x, y, z  
  
    q, x, y, z = paddle.jit.dy2static.convert_ifelse(  
        fluid.layers.mean(x)[0] < 5, true_fn_0, false_fn_0,  
        (x, y), (x, y), (q, x, y, z))  
    q = q + 1  
    n = q + 2  
    x = n  
    return x
```

# Transformer | 其它

| Transformer            | 转换逻辑                                                                                                                                                                                                                                                                            |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SplitAssignTransformer | 将多层赋值语句拆分为多个单对赋值表达式：<br>(1) <code>a, b = c, d</code> 拆为 <code>a = c</code> 与 <code>b = d</code><br>(2) <code>a = b = c</code> 拆为 <code>b = c</code> 与 <code>a = b</code><br>(3) <code>a, b = c, d = x</code> 拆为 <code>c, d = x</code> 、 <code>a = c</code> 、 <code>b = d</code> |
| AssertTransformer      | <code>assert</code> 语句改写为调用 <code>convert_assert</code> ，其将根据是否依赖tensor而转换为Assert算子或python <code>assert</code>                                                                                                                                                                  |
| PrintTransformer       | <code>print</code> 语句改写为调用 <code>convert_print</code> ，其将根据是否依赖tensor而转换为Print算子或python <code>print</code>                                                                                                                                                                      |
| CallTransformer        | 可调用对象改写为调用 <code>convert_call</code> ，其将对白名单（ <code>paddle api</code> 、 <code>lambda</code> 、内置函数、 <code>collections</code> 等， <code>len</code> 特殊处理）之外执行动转静转换                                                                                                                  |
| CastTransformer        | 强制类型转换改写为调用 <code>convert_var_dtype</code> ，当待转变量为tensor且目标类型为 <code>bool</code> 、 <code>int</code> 、 <code>float</code> 时添加cast算子，普通变量则调用python转换，否则报错                                                                                                                         |



# 个人小结

## ● 问题与不足：

1. 需要详读执行框架底层实现逻辑源码，以便今后心中有脉络
2. 深度学习基础知识欠缺，需要整理总结各领域常用的模型结构，有助于功能应用场景考虑周全
3. 对训练api的运算分类和常见用法不熟悉，需要日常多阅读api文档和模型库示例代码

## ● 建议

1. 官网文档很详细，但是主要面向使用者，对开发者而言框架功能设计和实现原理文档比较欠缺
2. 好记忆不如烂笔头，paddle功能迭代很快，熟悉需要日常逐渐积累，做好笔记方便温习



# Q&A

谢谢